ReactJS

Day - 2

Props

- Props are arguments passed into **React components**. Props are passed to components via HTML attributes. props stands for properties.

- Props in **React** are the objects used to pass data/information to a Component. It is short for Properties. It is responsible for passing the data ...

- Props is a special keyword in React that stands for properties and is used for passing data from one component to another. Data with props are passed in a unidirectional flow from parent to child.

React Components

Loaded: 100.00%

- The entire application is divided into a small logical group of code, which is known as components.
- Each component exists in the same space, but they work independently from one another and merge all in a parent component, which will be the final UI of the application.
- Every React component have their own structure, methods as well as APIs. They can be reusable as per the need.
- ReactJS has two types of components –
  1. Functional Components
  2. Class Components

Functional Components

- Functional components provide a way to write components that only contain a render method and don't have their own state.
- They are simply JavaScript functions that may or may not receive data as parameters.
- Create only a function that takes props(properties) as input and returns what should be rendered. A valid functional component is as shown in the below example.

```
function WelcomeMessage(props) {
    return <h1>Welcome to the , {props.name}</h1>;
}
```

- The functional component is also known as a **stateless component** because they do not hold or manage state. It can be explained in the below example.

Class Components

- Class component requires to extend from React.Component and create a render function returning a React element.
- Also a class can be defined by extending Component and has a render function.
- Data can be passed from one class to other class components.

```
class MyComponent extends React.Component {
  render() {
    return (
      <div>This is main component.</div>
    );
  }
}
```

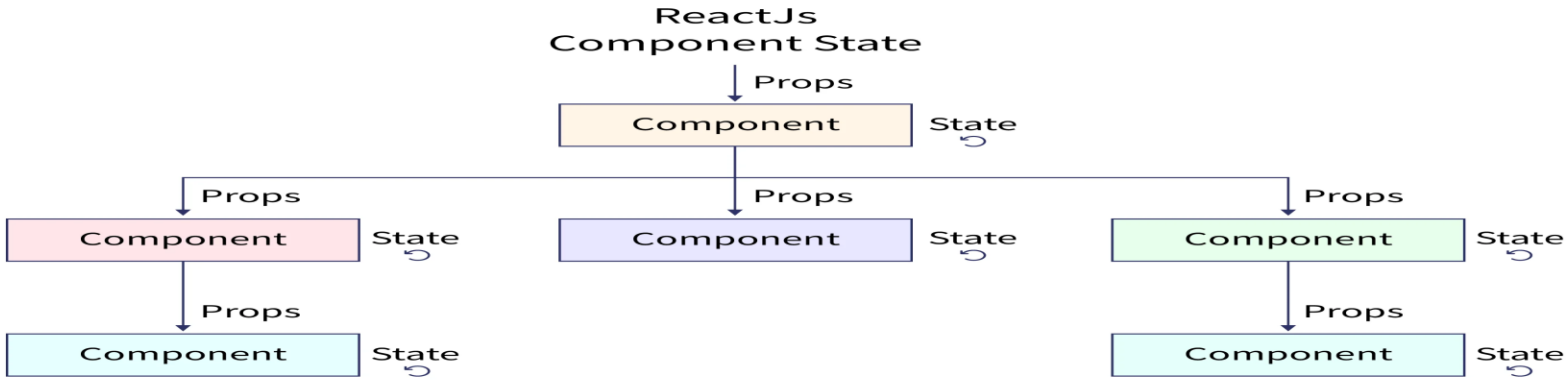
- The class component is also known as a **stateful component** because they can hold or manage local state.

contd..

- The components created of the two types are **equivalent**, and also the basic difference between a functional component and class component.
- **Functional component** is used only when the component does not require interacting or working with any other component i.e. these components do not require data from other components
- Multiple functional components can be composed under a single functional component.
- Class components can also be used for this purpose but it is not recommended if there is no need will make the application in-efficient.
- A class component can be called in a functional component in React. This is achieved by using the React Hooks API, specifically the useState and useEffect hooks.

React State

- The **state** in React is an instance of the **React Component Class** that can be defined as an object of a set of observable properties that control the behavior of the component.
- In other words, the **State** of **component** is an object that holds some information that may change over the lifetime of the component.
- **React component has built-in state object**. The state object is where store property values are stored that belong to component. **State** serves as container that encapsulates relevant data, logic, and behavior within a component.



contd..

- There is a constant **flow of dynamic data** thru components in react.
- React provides **state objects** to manage the dynamic nature of data, by **storing** and to **manipulate** the changing data of any component. State ensures re-rendering of the component in case of data change.
- **User Interface** in React applications thus is **function** of the **state**.
- **State** in **React** refers to **JS object** that holds the changing data of any components and also keeps track of those changes over time between different renders.
- As the data is dynamic nature of the state, it allows users to make dynamic and more interactive web apps. Using state to take inputs and capture changing data from API's.
- class components use the state property, while the new function component use hooks to manage the state.

State management in React

- **State management** is critical concept when working with React.
- React components can hold **local state**, but as applications grow, managing state across multiple components becomes complex.
- React provides several tools for state management – **Hooks, Context API,** and **Redux**.
  - ✓ **Local State (useState):** Manage data within a single component.
  - ✓ **Global State (Context API)**: Share state across multiple components.
  - ✓ **Centralised State (Redux)**: Manage complex state with a global store for large apps.
  - ✓ **Immutability**: State cannot be directly mutated; it must be updated via functions.
  - ✓ **Re-renders**: React re-renders components when state changes.

useReducer()

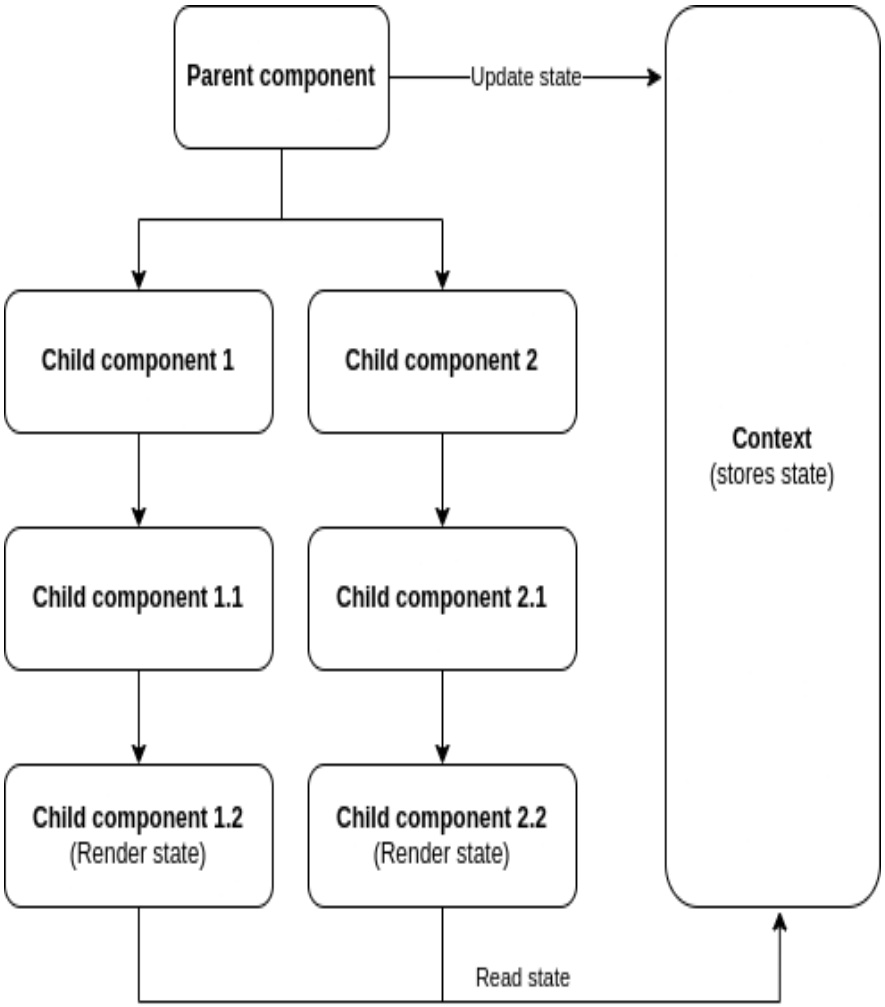
- **useReducer()** hook is used to store and update states. Syntax is –

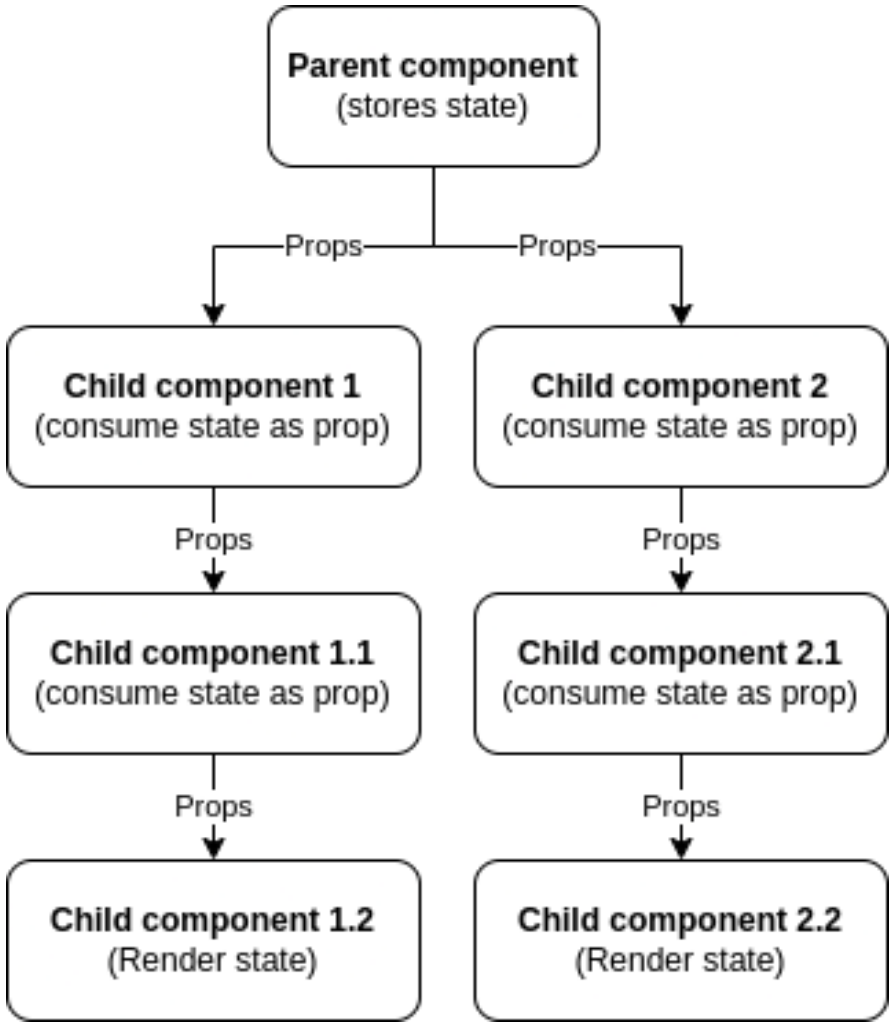
  const [state, dispatch] = useReducer(reducer, initialState);

  - ✓ **reducer:** A pure function that takes the current state and an action object, and returns a new state. It contains all the logic for state transitions, typically using a switch statement based on the action type.
  - ✓ **initialState:** The starting value of the state.
  - ✓ **state:** The current state value, which triggers a re-render when updated.
  - ✓ **dispatch:** A function used to dispatch an action object to the reducer, thus triggering a state update.
- **useReducer()** is used in scenarios where its needed managing more complex state logic in functional components. E.g. when the next state depends on the previous one, the state logic involves multiple sub-values, or when needed to centralize state update logic.

useContext()

- **React context** is a built-in feature that allows to share data, such as UI theme, authenticated user, or locale preferences, across component tree without having to pass props manually at every level.





contd..

- **useContext** is a React hook for subscribing to React Context, allowing functional components to access shared data (state,functions) from a Provider component without prop drilling, making data flow cleaner by reading values directly from the nearest Provider up the component tree.
- **useContext** takes context object (created with createContext) as argument and returns the current context value, simplifying global state management for things like themes, user auth, or preferences.
- **Create Context**: Use React.createContext() to create a Context object, defining its structure and default value (e.g., const ThemeContext = createContext('light');).
- **Provide Context**: Wrap the part of your component tree that needs the data with <Context.Provider value={...}>, passing the data through the value prop.
- **Consume Context**: In any descendant component, import useContext and call useContext(Context) to get the value.

contd..

- **ReactNode** is a type representing anything that React can render on the screen. It is the most inclusive and flexible type for content that may be displayed in your application, commonly used for defining the type of a component's children prop.

What a ReactNode includes

A ReactNode is a union type that can be one of the following:

- **React elements:** An object created via JSX (e.g., Hello).
- **Primitive types:**
  - string (text).
  - number.
  - boolean (though true and false are ignored and not rendered).
  - null or undefined (which are also ignored and do not render anything).
- **Arrays/Iterables:** An array containing any combination of other ReactNodes (for rendering lists of items).
- **Fragments and Portals:** Other specific React constructs like React.Fragment or React.Portal.

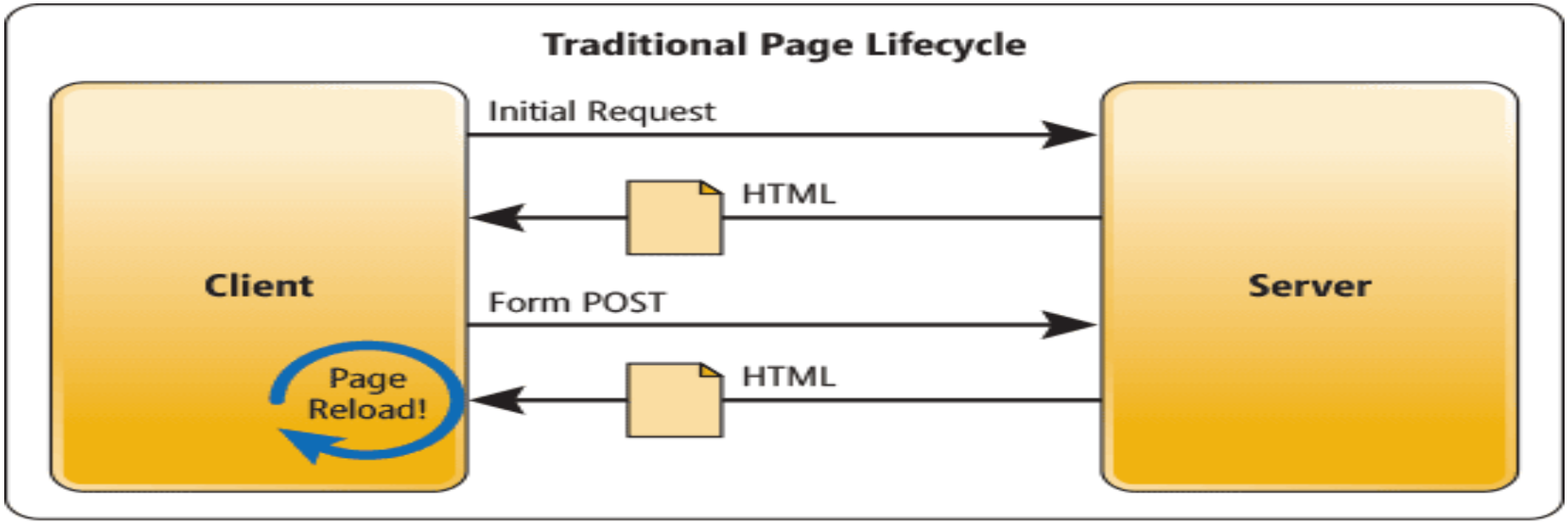
useCallback()

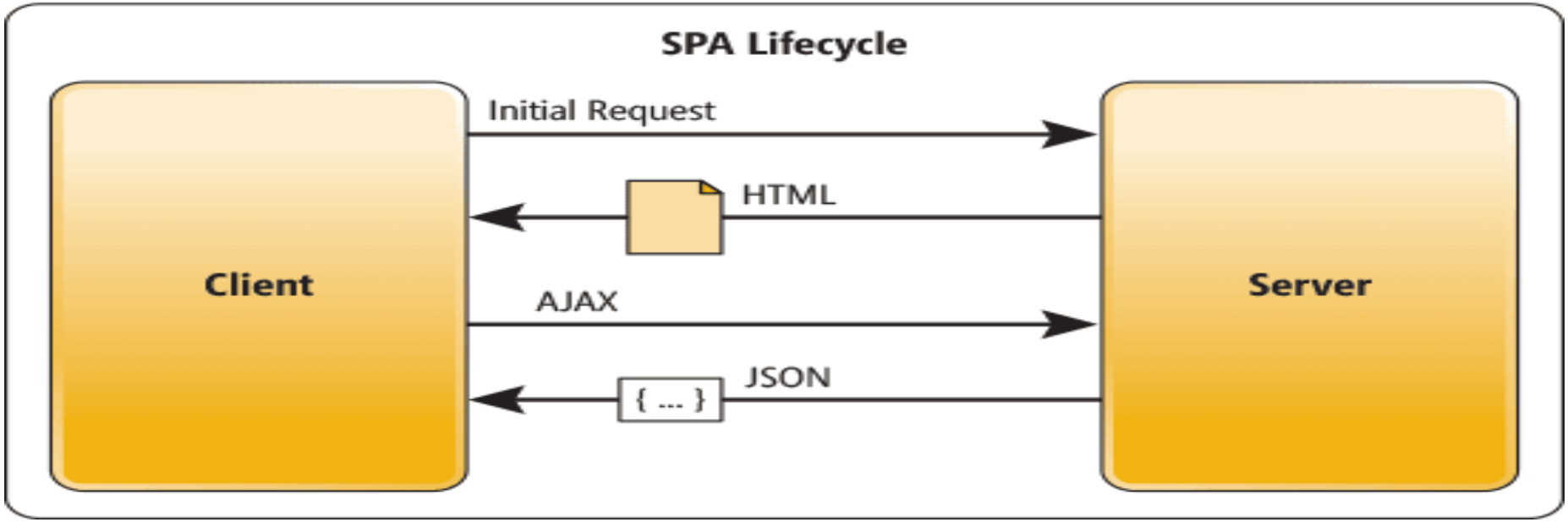
- **useCallback hook** returns a memoized function to reduce unnecessary callbacks. useCallback hook is used when there is component in which the child re-renders repeatedly without need.
- In JavaScript, functions are objects, and a new function instance (with a different memory reference) is created every time a component re-renders. This can cause performance issues, especially when functions are passed as props to child components, even if the function's logic is identical.
- **useCallback** prevents this by caching (memoizing) the function definition. React will return the same cached function instance on subsequent renders, only creating a new one if the values in the dependency array have changed.

useRef()

- **useRef** hook is a built-in function that provides a way to persist mutable values across component renders without causing a re-render.
- **useRef** returns a plain JavaScript object with a single, mutable property called current.
- **Persistence:** The value stored in ref.current persists for the entire lifetime of the component, unlike regular variables which are reset on every re-render.
- **No Re-renders:** Critically, updating the current property does not trigger a re-render of the component. This makes it ideal for storing values that don't directly affect the component's UI output.
- **Mutable:** The current property can be directly modified.
- **useRef**: Best for data that does not affect the UI, as updates do not trigger a re-render where as **useState** is designed for data that affects the UI, as updates trigger re-renders.

Single Page Application





contd..

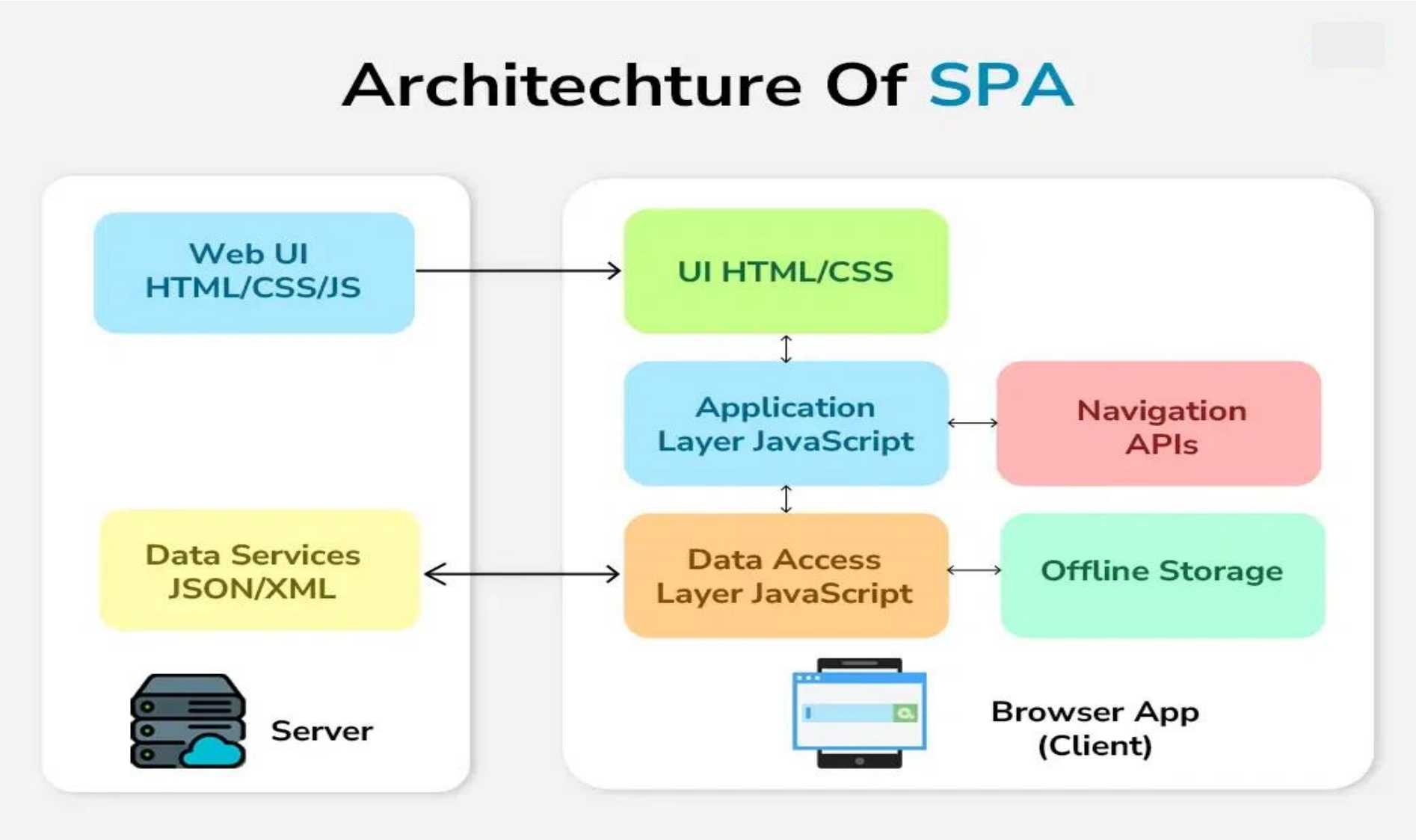
- **React Router** is a powerful and widely adopted library designed specifically for routing in React applications.
- **React Router** empowers to build a navigation structure within SPA that mimics the behavior of a traditional multi-page website, all while staying within the confines of a single HTML page.
- **React Router** provides developers to
  a. define routes,
  b. map them to specific components, and
  c. seamlessly manage transitions between different views or sections of a web application without requiring full page reloads.
- End results is faster loading times, improved user experience, and a more fluid and dynamic feel for the end-user.

contd..

❖ **Core Components of React Router:**

- ✓ **BrowserRouter:** The foundation for web applications, enabling client-side routing using the HTML5 History API (pushState, replaceState, and the popstate event).
- ✓ **Routes:** Acts as a parent container, rendering the appropriate route based on the current URL.
- ✓ **Route:** Defines a route mapping, specifying a component to render when the URL matches a defined path.
- ✓ **Link:** Provides declarative navigation, allowing users to transition between routes without manual URL manipulation.
- ✓ **useNavigate Hook:** A hook that lets you programmatically navigate within your application.
- ✓ **useParams Hook:** A hook for accessing URL parameters declared within your routes.

contd..


Architechture Of SPA
Web UI
HTML/CSS/JS
UI HTML/CSS
Application
Layer JavaScript
Navigation
APIs
Data Services
JSON/XML
Data Access
Layer JavaScript
Offline Storage
Server
Browser App
(Client)

Axios

- **Axios** is a very popular **JavaScript library** user can use to perform HTTP requests to external resources, that work in both **Browser** and **Node.js** platforms
- **Axios** returns information from the backend in a Promise object like fetch(). It requires installation and importing to your code
- **Axios** automatically stringifies the data when sending requests, and the data Axios returns is the data, without the need to convert first to JSON.
- **Axios** allows user to make HTTP requests to APIs and other external resources.
- **Axios** library for making HTTP requests in JavaScript applications because it provides a **simple API** and supports **Promises**.